SPRINT 2 · BIENVENIDA

JavaScript y Gestión de Tareas

Proyecto Individual · Planificador de Tareas

NUEVO SPRINT, NUEVO NIVEL DE INTERACCIÓN

Has cerrado el Sprint 1 y ya cuentas con la primera versión de la interfaz. Ahora continuarás trabajando sobre ese mismo proyecto para comenzar a darle mayor comportamiento mediante JavaScript.

Bienvenido al Sprint 2

En este nuevo sprint comenzarás a transformar la interfaz que construiste en el Sprint 1 en una aplicación con mayor interacción. La idea no es comenzar desde cero, sino aprovechar lo que ya construiste y organizar progresivamente la lógica de tu aplicación.

Durante este sprint trabajarás con estructuras de JavaScript que te permitirán representar y gestionar las tareas de tu aplicación. Este será un paso importante para preparar el proyecto para las funcionalidades que desarrollarás más adelante.

Objetivo del Sprint 2

Comenzar a gestionar la información de las tareas mediante JavaScript, utilizando clases, objetos y estructuras que permitan organizar el comportamiento de la aplicación.

Tareas que desarrollarás

Tarea 4 Crear la Clase de Tareas utilizando JavaScript Implementar interacción marcando una tarea como completada	**Tarea 5** Agregar Tareas de manera Programada	**Tarea 6** Eliminar una Tarea

RECUERDA

Este sprint continúa el proyecto que ya comenzaste. Conserva tu repositorio, tu historial de commits y la estructura construida durante el Sprint 1.

SPRINT 2 · TAREA 4

Crear la Clase de Tareas utilizando JavaScript

Duración estimada: 1 hora y 45 minutos

OBJETIVO DE LA TAREA

Parte 1: Implementarás una clase de JavaScript que represente y gestione las tareas de la aplicación mediante una estructura de objetos.

Parte 2: Implementarás la interacción que permita marcar y desmarcar una tarea como completada

Resultado esperado - PARTE 1

Al finalizar esta actividad deberás contar con una clase `TaskManager` preparada para almacenar las tareas de la aplicación.

- Una carpeta `js` organizada.
- Los archivos JavaScript correctamente vinculados.
- Una clase `TaskManager` creada en `taskManager.js`.
- Una propiedad `tasks` inicializada como un array vacío.
- Una instancia de `TaskManager` verificada desde la consola del navegador.

Resultado esperado - PARTE 2

Permitir que el usuario pueda marcar y desmarcar una tarea como completada, proporcionando una respuesta visual que permita identificar fácilmente su estado. Elegir entre estas opciones para la implementación

- Un cambio de color o fondo.
- Un cambio en el borde de la tarjeta.
- Tachado del nombre de la tarea.
- Un indicador o ícono de completado.
- Un cambio visual en el estado.
- Una combinación de varios elementos.

Contexto

En el Sprint 1 la información de las tareas estaba principalmente representada mediante HTML. En este sprint comenzaremos a representar esa información mediante estructuras de JavaScript.

CONCEPTO CLAVE

La clase `TaskManager` será la base que utilizaremos en las siguientes tareas para administrar las tareas de la aplicación.

Herramientas

HTML5	Bootstrap 5	JavaScript

DESARROLLO DE LA ACTIVIDAD – PARTE 1

Paso 1 · Configurar la estructura del proyecto

Antes de crear la clase, reorganizarás la estructura de tus archivos JavaScript. Esta organización prepara el proyecto para separar responsabilidades a medida que la aplicación crezca.

1

Crear la carpeta `js`

Si todavía no existe, crea una carpeta llamada `js` dentro de tu proyecto.

2

Organizar `index.js`

Copia o mueve el archivo JavaScript existente a la carpeta `js` y asegúrate de que se llame `index.js`.

3

Actualizar la ruta del script

En `index.html`, actualiza la etiqueta `<script>` para que apunte a `js/index.js`.

4

Crear `taskManager.js`

Dentro de la carpeta `js`, crea un nuevo archivo llamado `taskManager.js`.

5

Actualizar el orden de los scripts

Agrega el script de `taskManager.js` antes del script de `index.js`.

¿POR QUÉ HACEMOS ESTO?

Estamos preparando una estructura más ordenada para el código JavaScript. En las siguientes tareas iremos incorporando nuevas responsabilidades a estos archivos.

Referencia de la estructura

```
Planificador-Tareas/
├── css/
│   └── styles.css
├── js/
│   ├── index.js
│   └── taskManager.js
└── index.html
```

Orden de carga en `index.html`

```
<script src="js/taskManager.js"></script>
<script src="js/index.js"></script>
```

> **IMPORTANTE**
>
> `taskManager.js` debe cargarse antes que `index.js`, porque en la siguiente etapa `index.js` utilizará la clase que acabas de crear.

Paso 2 · Crear la clase TaskManager

Ahora crearás la clase que será responsable de gestionar las tareas de la aplicación.

1. Crear la clase

En `js/taskManager.js`, crea una clase llamada `TaskManager`.

```
class TaskManager {
    constructor() {
        this.tasks = [];
    }
}
```

2. Inicializar la colección de tareas

Dentro del constructor, inicializa la propiedad `this.tasks` como un array vacío. Por ahora no agregaremos métodos ni tareas; el objetivo es crear la estructura básica que utilizaremos posteriormente.

> **¿QUÉ REPRESENTA `THIS.TASKS`?**
>
> Es la colección donde la instancia de `TaskManager` almacenará las tareas de la aplicación. En este punto comienza vacía.

3. Probar la clase desde `index.js`

En `js/index.js`, crea una nueva instancia de `TaskManager` y muestra su propiedad `tasks` en la consola.

```
const taskManager = new TaskManager();

console.log(taskManager.tasks);
```

Resultado esperado

Al abrir la consola del navegador deberías visualizar un array vacío:

```
[]
```

SI APARECE UN ERROR

Revisa principalmente la ubicación de los archivos, las rutas de los `<script>`, el orden de carga, el nombre de la clase y los nombres de los archivos.

PRUEBA Y COMPRENSIÓN

Prueba tu código

Antes de continuar, verifica que la clase y la estructura de archivos estén funcionando correctamente.

Pasos de comprobación

- Abre `index.html` en tu navegador.
- Abre las herramientas de desarrollador.
- Ingresa a la pestaña Console.
- Verifica que aparezca un array vacío: `[]`.

¿Qué acabas de construir?

Aplicación	TaskManager	tasks[]	Tareas

La clase `TaskManager` representa una primera estructura para organizar las tareas desde JavaScript. En las siguientes actividades esta estructura comenzará a utilizarse para agregar y gestionar información.

Verificación rápida

☐ ¿La consola muestra `[]`?

☐ ¿taskManager es una instancia de TaskManager?

☐ ¿`this.tasks` está inicializado como un array?

☐ ¿Los scripts se cargan en el orden correcto?

NO AVANCES TODAVÍA

En esta tarea no necesitas crear tareas desde el formulario ni agregar métodos para eliminarlas. Esas funcionalidades pertenecen a las siguientes tareas del Sprint 2.

DESARROLLO DE LA ACTIVIDAD – PARTE 2

Objetivo

Agregar una interacción visual a cada tarjeta de tarea que permita marcarla como completada y volver a marcarla como pendiente. El cambio debe realizarse directamente en la interfaz, sin recargar la página.

Requerimiento

Cada tarjeta de tarea deberá incluir un botón que permita cambiar su estado visual:

- Al hacer clic, la tarea se marca como completada.
- La tarjeta debe mostrar un cambio visual que permita identificar que está completada.
- El mismo botón debe permitir desmarcarla y devolverla a su estado pendiente.
- La interacción debe realizarse sin recargar la página.

Comportamiento esperado

El botón funciona como un interruptor (toggle): cada clic cambia entre los estados “Pendiente” y “Completada”. Puedes utilizar clases de Bootstrap y JavaScript para modificar la apariencia de la tarjeta.

Criterios de aceptación

- ✓ La tarjeta contiene un botón para marcar y/o desmarcar la tarea.
- ✓ Al marcar una tarea como completada, existe un cambio visual evidente en la tarjeta.
- ✓ Al volver a hacer clic, la tarjeta recupera su apariencia de tarea pendiente.
- ✓ La interacción funciona sin recargar la página.
- ✓ La solución utiliza JavaScript para controlar el comportamiento

CIERRE · TAREA 4

Verificación y entregable

Checklist de la Tarea 4

Estructura ☐ Existe la carpeta `js`. ☐ `index.js` está dentro de `js`. ☐ Existe `taskManager.js`. ☐ Las rutas de los scripts son correctas.	**TaskManager** ☐ Existe la clase `TaskManager`. ☐ El constructor inicializa `this.tasks = []`. ☐ No se agregaron métodos innecesarios.
Prueba ☐ Se creó una instancia de `TaskManager`. ☐ La propiedad `tasks` se muestra en consola. ☐ La consola muestra `[]`. ☐ El cambio en UI del estado de la tarea funciona	**Proyecto** ☐ Los cambios fueron guardados. ☐ El repositorio refleja el avance de la tarea. ☐ El historial de commits continúa evidenciando tu progreso.

Entregable

Al finalizar la actividad, conserva tu proyecto actualizado en el repositorio de GitHub. La revisión considerará el avance realizado sobre el proyecto existente y la trazabilidad de tu trabajo.

> **RESULTADO ESPERADO**
>
> Has creado la estructura básica de `TaskManager` y una colección `tasks` lista para comenzar a gestionar las tareas de la aplicación. Tienes un cambio de estado funcional para las tareas creadas previamente

¿Qué sigue?

En las siguientes tareas del Sprint 2 utilizarás esta estructura para comenzar a trabajar con las tareas de la aplicación. La clase que acabas de crear será la base para incorporar nuevas funcionalidades sin perder la organización del proyecto.

> **CONTINÚA CONSTRUYENDO SOBRE LO QUE YA TIENES**
>
> Cada tarea agrega una nueva capacidad al mismo proyecto. No crees un proyecto nuevo: evoluciona el que comenzaste en el Sprint 1.